



Machine Learning

Chapitre 4: Apprentissage non-supervisé

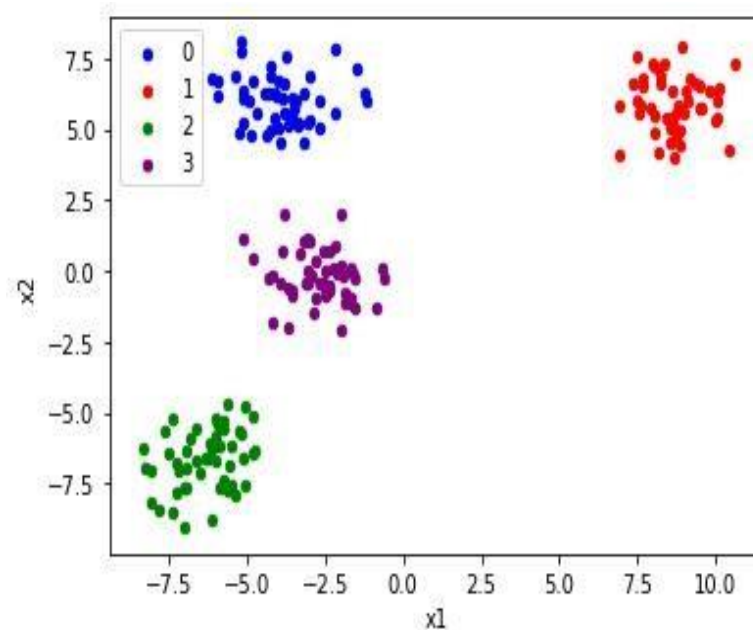
Mme BEDAD Fatima
fatima.bedad@univ-temouchent.edu.dz

2023-2024

Unsupervised learning

1. Clustering

Clustering is the most popular technique in unsupervised learning where **data is grouped based on the similarity of the data- points** . Clustering has many real- life applications where it can be used in a variety of situations.



1. Clustering

Method name	Parameters	Scalability	Usecase	Geometry (metric used)
K-Means	number of clusters	Very large <code>n_samples</code> , medium <code>n_clusters</code> with MiniBatch code	General-purpose, even cluster size, flat geometry, not too many clusters	Distances between points
Affinity propagation	damping, sample preference	Not scalable with <code>n_samples</code>	Many clusters, uneven cluster size, non-flat geometry	Graph distance (e.g. nearest-neighbor graph)
Mean-shift	bandwidth	Not scalable with <code>n_samples</code>	Many clusters, uneven cluster size, non-flat geometry	Distances between points
Spectral clustering	number of clusters	Medium <code>n_samples</code> , small <code>n_clusters</code>	Few clusters, even cluster size, non-flat geometry	Graph distance (e.g. nearest-neighbor graph)
Ward hierarchical clustering	number of clusters or distance threshold	Large <code>n_samples</code> and <code>n_clusters</code>	Many clusters, possibly connectivity constraints	Distances between points
Agglomerative clustering	number of clusters or distance threshold, linkage type, distance	Large <code>n_samples</code> and <code>n_clusters</code>	Many clusters, possibly connectivity constraints, non Euclidean distances	Any pairwise distance
DBSCAN	neighborhood size	Very large <code>n_samples</code> , medium <code>n_clusters</code>	Non-flat geometry, uneven cluster sizes	Distances between nearest points
OPTICS	minimum cluster membership	Very large <code>n_samples</code> , large <code>n_clusters</code>	Non-flat geometry, uneven cluster sizes, variable cluster density	Distances between points
Gaussian mixtures	many	Not scalable	Flat geometry, good for density estimation	Mahalanobis distances to centers
Birch	branching factor, threshold, optional global clusterer.	Large <code>n_clusters</code> and <code>n_samples</code>	Large dataset, outlier removal, data reduction.	Euclidean distance between points

1. Clustering - K-means

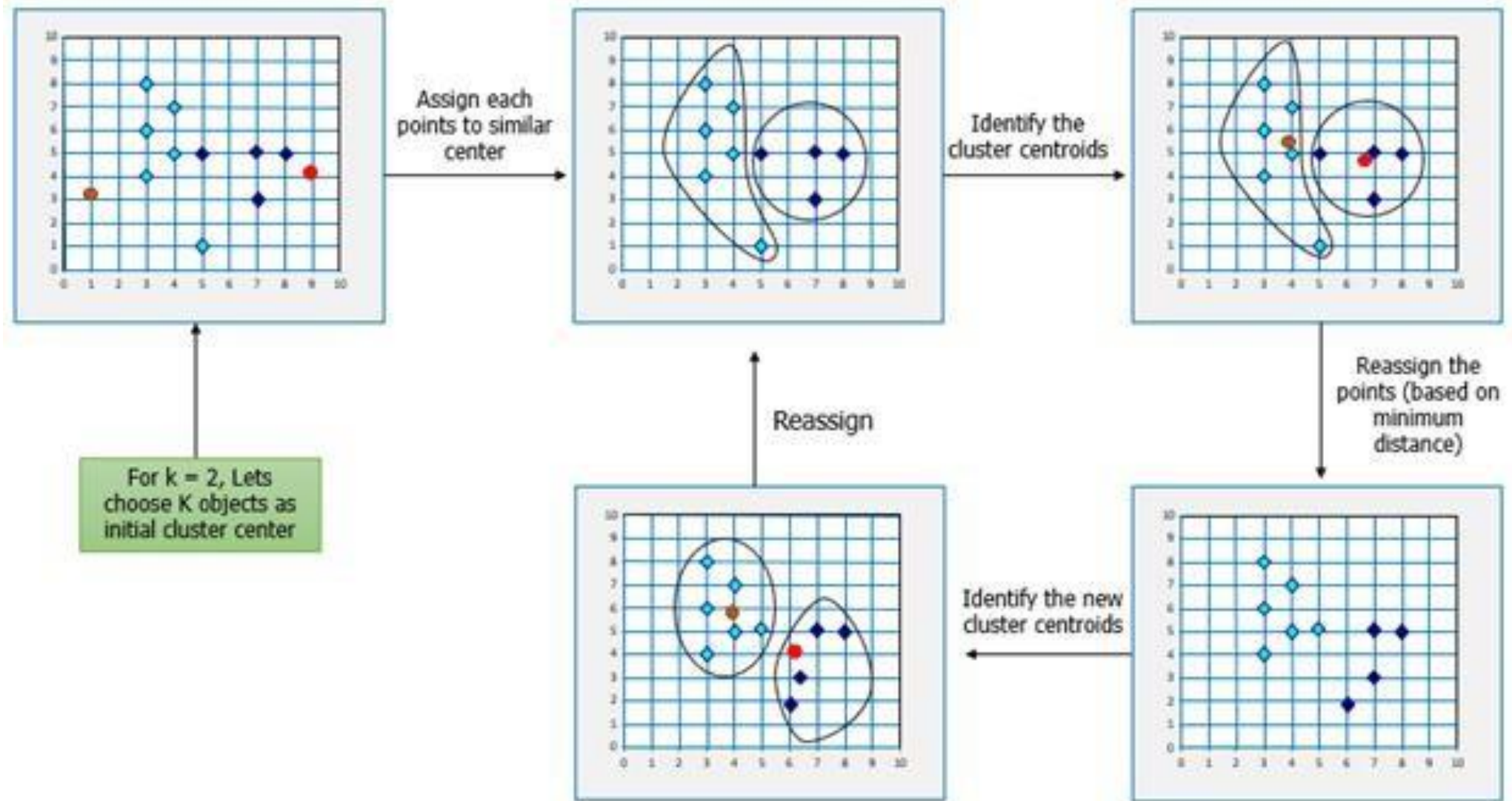
The KMeans algorithm clusters data by trying to separate samples in **n groups** of equal variance, minimizing a criterion known as the **inertia or within-cluster sum-of-squares**. This algorithm **requires the number of clusters to be specified**. It scales well to large number of samples and has been used across a large range of application areas in many different fields.

The k-means algorithm divides a set of **N** samples **X** into **K** disjoint clusters **C**, each described by the mean **μ_j** of the samples in the cluster. The means are commonly called the cluster **“centroids”**; note that they are not, in general, points from **X**, although they live in the same space.

The K-means algorithm aims to choose centroids that minimise **the inertia, or within-cluster sum-of-squares criterion**:

$$\sum_{i=0}^n \min_{\mu_j \in C} (||x_i - \mu_j||^2)$$

1. Clustering - K-means



1. Clustering - K-means

Clustering performance evaluation :

Clustering	
'adjusted_mutual_info_score'	<code>metrics.adjusted_mutual_info_score</code>
'adjusted_rand_score'	<code>metrics.adjusted_rand_score</code>
'completeness_score'	<code>metrics.completeness_score</code>
'fowlkes_mallows_score'	<code>metrics.fowlkes_mallows_score</code>
'homogeneity_score'	<code>metrics.homogeneity_score</code>
'mutual_info_score'	<code>metrics.mutual_info_score</code>
'normalized_mutual_info_score'	<code>metrics.normalized_mutual_info_score</code>
'v_measure_score'	<code>metrics.v_measure_score</code>

1. Clustering - K-means

Clustering performance evaluation :

```
from sklearn import metrics
```

```
labels_true = [0, 0, 0, 1, 1, 1]
```

```
labels_pred = [0, 0, 1, 1, 2, 2]
```

```
metrics.adjusted_rand_score(labels_true, labels_pred)
```

Centroid initialization methods :

```
good_init = np.array([[-3, 3], [-3, 2], [-3, 1], [-1, 2], [0, 2]])
```

```
kmeans = KMeans(n_clusters=5, init=good_init)
```


7. Clustering - K-means

```
import matplotlib.pyplot as plt
import seaborn as sns; sns.set() # for plot styling
import numpy as np
from sklearn.datasets.samples_generator import make_blobs

# n_featuresint, optional (default=2)
X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=1.5, random_state=100)
print("Examples = ", X.shape)
# Visualize the Data
fig, ax = plt.subplots(figsize=(8,5))
ax.scatter(X[:, 0], X[:, 1], s=50)
ax.set_title("Visualize the Data")
ax.set_xlabel("X")
ax.set_ylabel("Y")
plt.show()
# Create Clusters
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=4)
kmeans.fit(X)
y_kmeans = kmeans.predict(X)
print("the 4 centroids that the algorithm found:
\n", kmeans.cluster_centers_)
# Visualize the results
fig, ax = plt.subplots(figsize=(8,5))
ax.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis')
centers = kmeans.cluster_centers_
ax.scatter(centers[:, 0], centers[:, 1], c='black', s=200, alpha=0.5);
ax.set_title("Visualize the results")
ax.set_xlabel("X")
ax.set_ylabel("Y")
plt.show()
```

```
# Assign new instances to the cluster whose centroid is closest:
X_new = np.array([[0, 2], [3, 2], [-3, 3], [-3, 2.5]])

y_kmeans_new = kmeans.predict(X_new)

# The transform() method measures the distance from each instance
to every centroid:

print("The distance from each instance to every centroid:

\n", kmeans.transform(X_new))
# Visualize the new results

fig, ax = plt.subplots(figsize=(8,5))

ax.scatter(X_new[:, 0], X_new[:, 1], c=y_kmeans_new, s=50,
cmap='viridis')

centers = kmeans.cluster_centers_

ax.scatter(centers[:, 0], centers[:, 1], c='black', s=200, alpha=0.5);
ax.set_title("Visualize the new results")

ax.set_xlabel("X")
ax.set_ylabel("Y")
plt.show()

print("inertia : ", kmeans.inertia_)
print("score : ", kmeans.score(X))
```

1. Clustering - Density-based spatial clustering of applications with noise (DBSCAN)

The **DBSCAN** algorithm should be used to find **associations** and **structures** in data that are hard to find manually but that can be relevant and useful to find **patterns** and **predict trends**. It defines clusters as continuous regions of high density.

The **DBSCAN** algorithm basically **requires 2 parameters**:

eps: specifies how close points should be to each other to be considered a part of a cluster. It means that **if the distance between two points is lower or equal to this value (eps), these points are considered neighbors**.

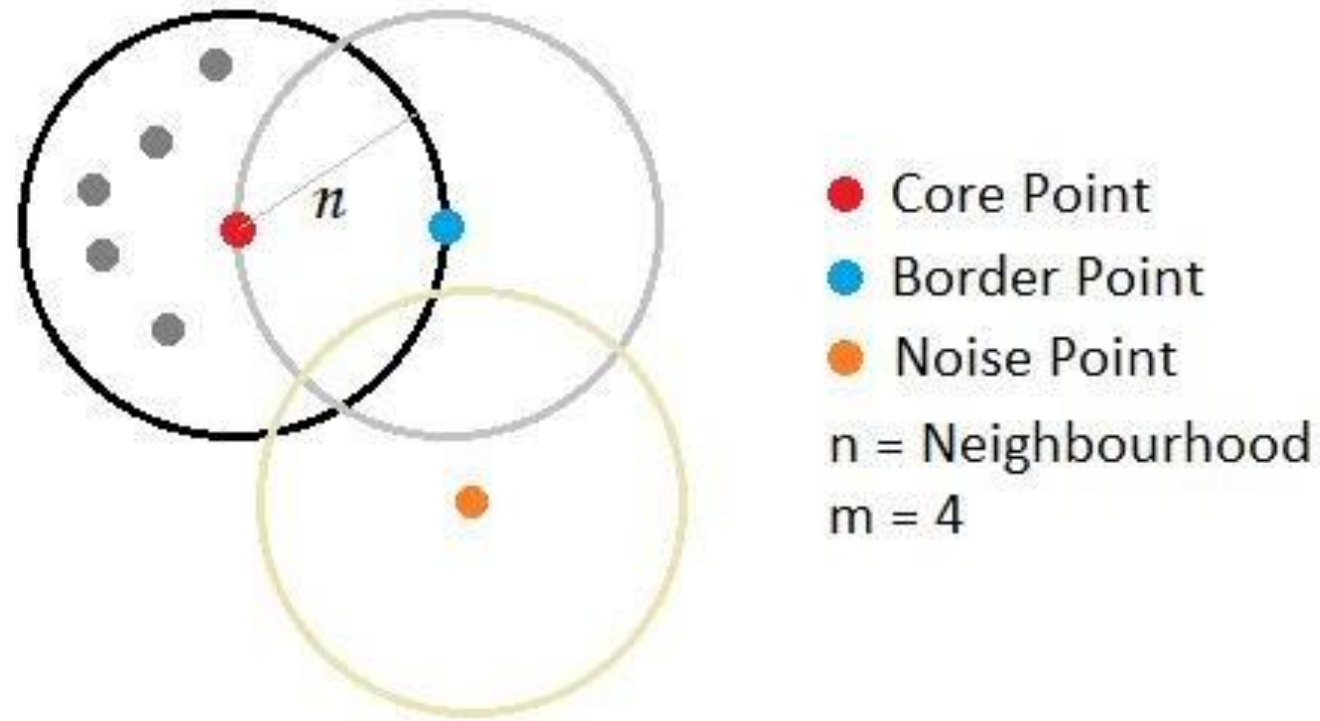
min_samples: **the minimum number of points to form a dense region**. For example, if we set the min_samples parameter as 5, then we need at least 5 points to form a dense region,

1. Clustering - Density-based spatial clustering of applications with noise (DBSCAN)

Here is how it works:

- 1) For each instance, the algorithm counts how many instances are located within a small distance ϵ (epsilon) from it. This region is called the instance's ϵ -neighborhood.
- 2) If an instance has at least `min_samples` instances in its ϵ -neighborhood (including itself), then it is considered a **core instance**. In other words, **core instances** are those that are located in dense regions.
- 3) All instances in the neighborhood of a **core instance** belong to the same **cluster**. This neighborhood may include other core instances; therefore, a long sequence of neighboring core instances forms a single cluster.
- 4) Any instance that is not a core instance and does not have one in its neighborhood is considered an **anomaly**.

1. Clustering - Density-based spatial clustering of applications with noise (DBSCAN)

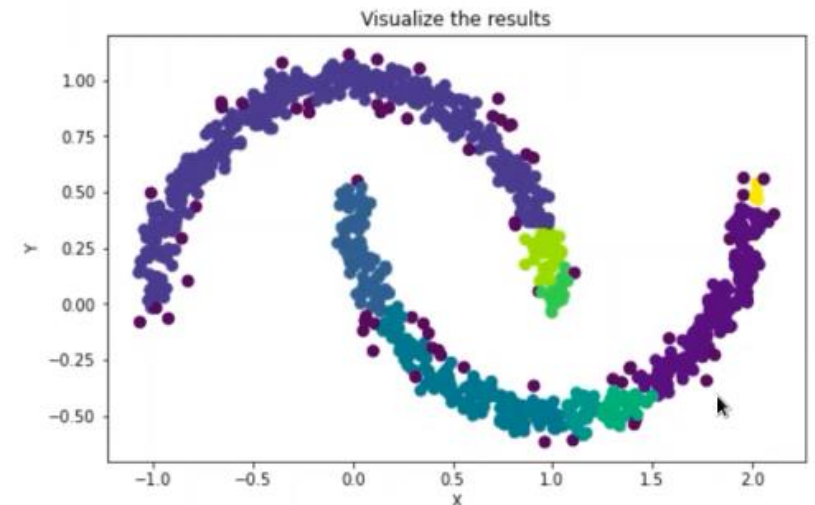


DBSCAN CLUSTERING

Abhijit Annaldas

1. Clustering - Density-based spatial clustering of applications with noise (DBSCAN)

```
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
X, y = make_moons(n_samples=1000, noise=0.05)
dbscan = DBSCAN(eps=0.05, min_samples=5)
dbscan.fit(X)
# The labels of all the instances. Notice that some instances have a cluster index equal to -1, which means that
# they are considered as anomalies by the algorithm.
print("The labels of all the instances : \n",dbscan.labels_)
# The indices of the core instances
print("The indices of the core instances : \n Len = ",len(dbscan.core_sample_indices_), "\n",
dbscan.core_sample_indices_)
# The core instances
print("the core instances : \n", dbscan.components_)
# Visualize the results
fig, ax = plt.subplots(figsize=(8,5))
ax.scatter(X[:, 0], X[:, 1], c= dbscan.labels_, s=50, cmap='viridis')
ax.set_title("Visualize the results")
ax.set_xlabel("X")
ax.set_ylabel("Y")
plt.show()
```



1. Clustering - Hierarchical clustering

Hierarchical clustering is a general family of clustering algorithms that build **nested clusters** by **merging** or **splitting** them successively. This hierarchy of clusters is represented as a **tree** (or **dendrogram**). **The root of the tree is the unique cluster** that gathers all the samples, the leaves being the clusters with only one sample.

The **AgglomerativeClustering** object performs a hierarchical clustering using a bottom up approach: each observation starts in its own cluster, and clusters are successively merged together.

1. Clustering - Hierarchical clustering

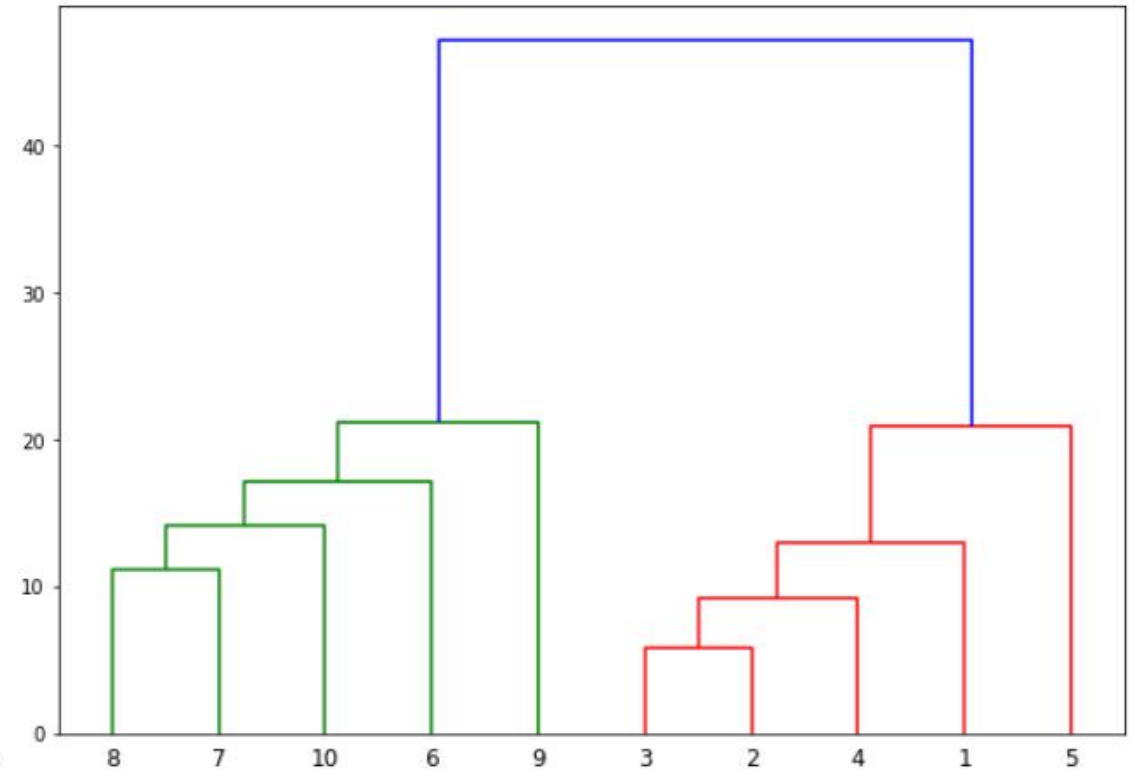
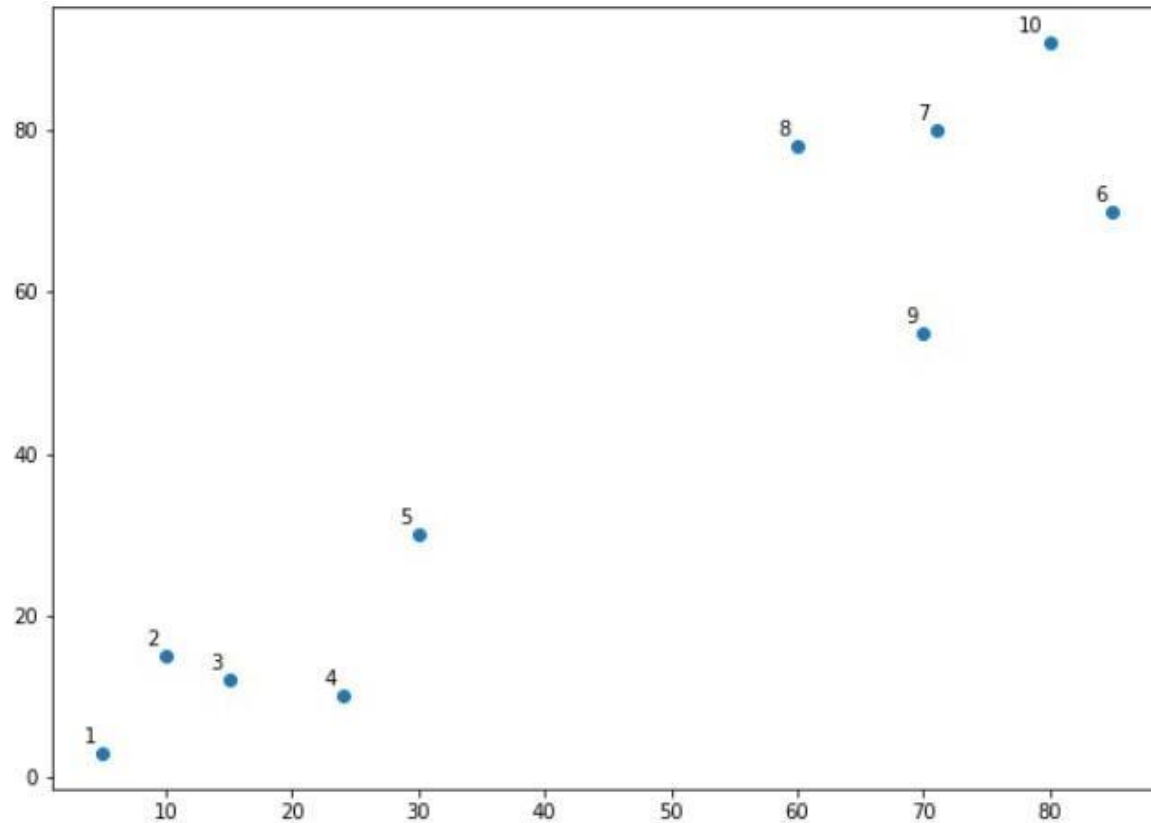
- Steps to Perform Hierarchical Clustering (agglomerative clustering):
 - 1) At the start, treat each data point as one cluster. Therefore, the number of clusters at the start will be K , while K is an integer representing the number of data points.
 - 2) Form a cluster by joining the two closest data points resulting in $K-1$ clusters.
 - 3) Form more clusters by joining the two closest clusters resulting in $K-2$ clusters.
 - 4) Repeat the above three steps until one big cluster is formed.
 - 5) Once single cluster is formed, dendrograms are used to divide into multiple clusters depending upon the problem.

1. Clustering - Hierarchical clustering

There are different ways to find distance between the clusters. The distance itself can be **Euclidean** or **Manhattan** distance. Following are some of the options to measure distance between two clusters:

- 1) Measure the distance between the closes points of two clusters.
- 2) **Measure the distance between the farthest points of two clusters.**
- 3) Measure the distance between the centroids of two clusters.
- 4) **Measure the distance between all possible combination of points between the two clusters and take the mean.**

1. Clustering - Hierarchical clustering



1. Clustering - Hierarchical clustering

```
X = np.array([[5,3],[10,15],[15,12],[24,10],[30,30],
             [85,70],[71,80],[60,78],[70,55],[80,91],])
"""
n_clustersint or None, default=2
The number of clusters to find. It must be None if distance_threshold is not None.

distance_thresholdfloat, default=None
The linkage distance threshold above which, clusters will not be merged.
If not None, n_clusters must be None and compute_full_tree must be True.
"""
model = AgglomerativeClustering(distance_threshold=0, n_clusters=None)
model.fit(X)

print("-----")
print(model.n_clusters_)
print(model.n_connected_components_)
print("-----")

plt.figure(figsize=(8, 6))
plt.title("Hierarchical Clustering Dendrogram")
# plot the top p levels of the dendrogram
plot_dendrogram(model, truncate_mode="level", p=1)
plt.xlabel("Number of points in node (or index of point if no parenthesis).")
plt.show()
```

1. Clustering - Hierarchical clustering

```
import scipy.cluster.hierarchy as shc
plt.figure(figsize=(8, 6))
plt.title("Hierarchical Clustering Dendrogram")
dend = shc.dendrogram(shc.linkage(X, method="ward"))
```

1. Clustering - Hierarchical clustering

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.cluster.hierarchy import dendrogram
from sklearn.cluster import AgglomerativeClustering

def plot_dendrogram(model, **kwargs):
    # Create linkage matrix and then plot the dendrogram
    # create the counts of samples under each node
    counts = np.zeros(model.children_.shape[0])
    n_samples = len(model.labels_)
    for i, merge in enumerate(model.children_):
        current_count = 0
        for child_idx in merge:
            if child_idx < n_samples:
                current_count += 1 # leaf node
            else:
                current_count += counts[child_idx - n_samples]
        counts[i] = current_count
    linkage_matrix = np.column_stack([model.children_, model.distances_,
                                     counts]).astype(float)

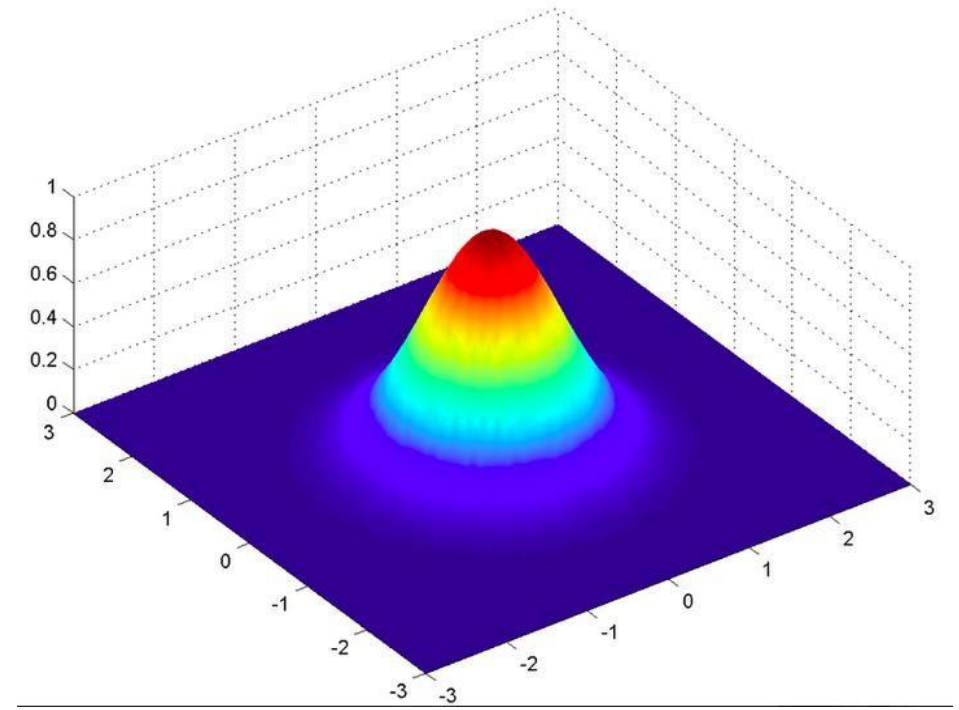
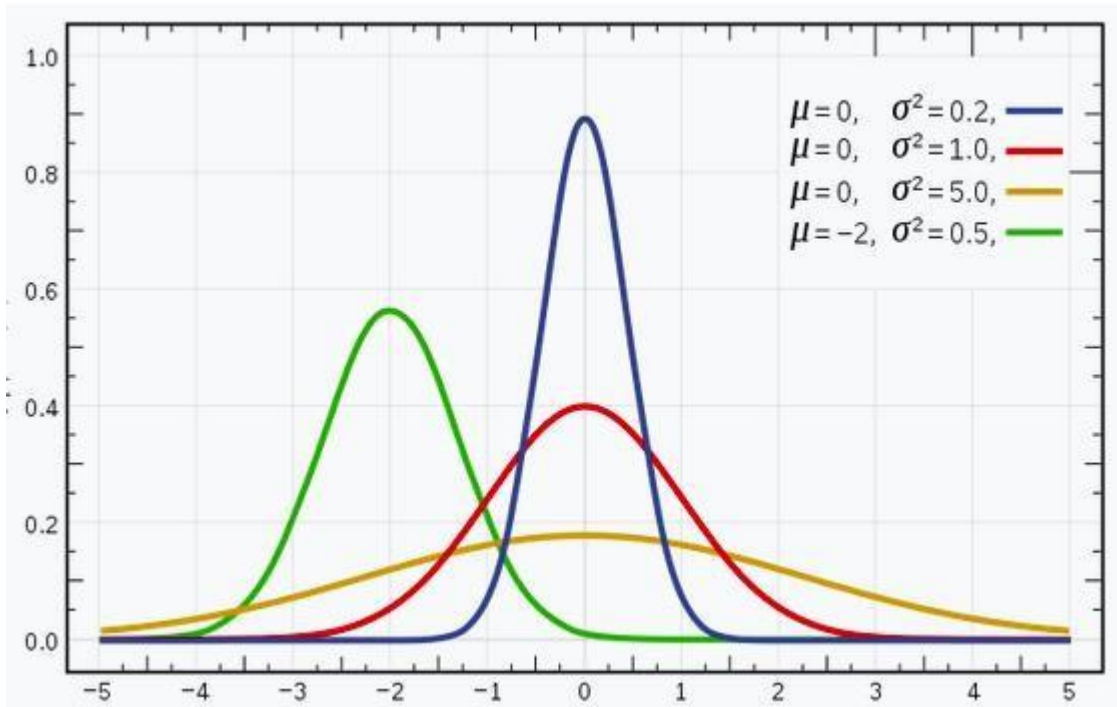
    # Plot the corresponding dendrogram
    dendrogram(linkage_matrix, **kwargs)
```

2. Gaussian Mixture

Gaussian Mixture Models (GMMs) assume that there are a certain number of **Gaussian distributions**, and each of these distributions represent a cluster. Hence, **a Gaussian Mixture Model tends to group the data points belonging to a single distribution together.**

8. Gaussian Mixture

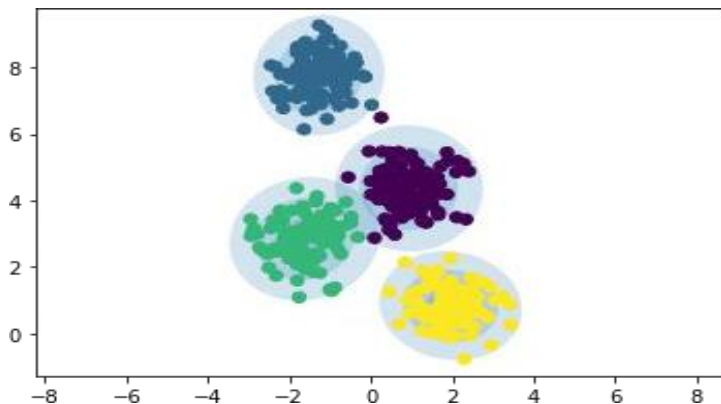
The Gaussian Distribution: The below image (right) has a few Gaussian distributions with a difference in mean (μ) and variance (σ^2). Remember that the higher the σ value more would be the spread:



2. Gaussian Mixture

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets.samples_generator import make_blobs
X, y_true = make_blobs(n_samples=400, centers=4,
                       cluster_std=0.60, random_state=0)

# training gaussian mixture model
from sklearn.mixture import GaussianMixture
gm = GaussianMixture(n_components=4)
gm.fit(X)
print("The weights of each mixture components:\n",gm.weights_)
print("The mean of each mixture component:\n",gm.means_)
print("The covariance of each mixture component:\n",gm.covariances_)
print("check whether or not the algorithm converged: ", gm.converged_)
print("nbr of iterations: ",gm.n_iter_)
#print("score: \n",gm.score_samples(X))
```



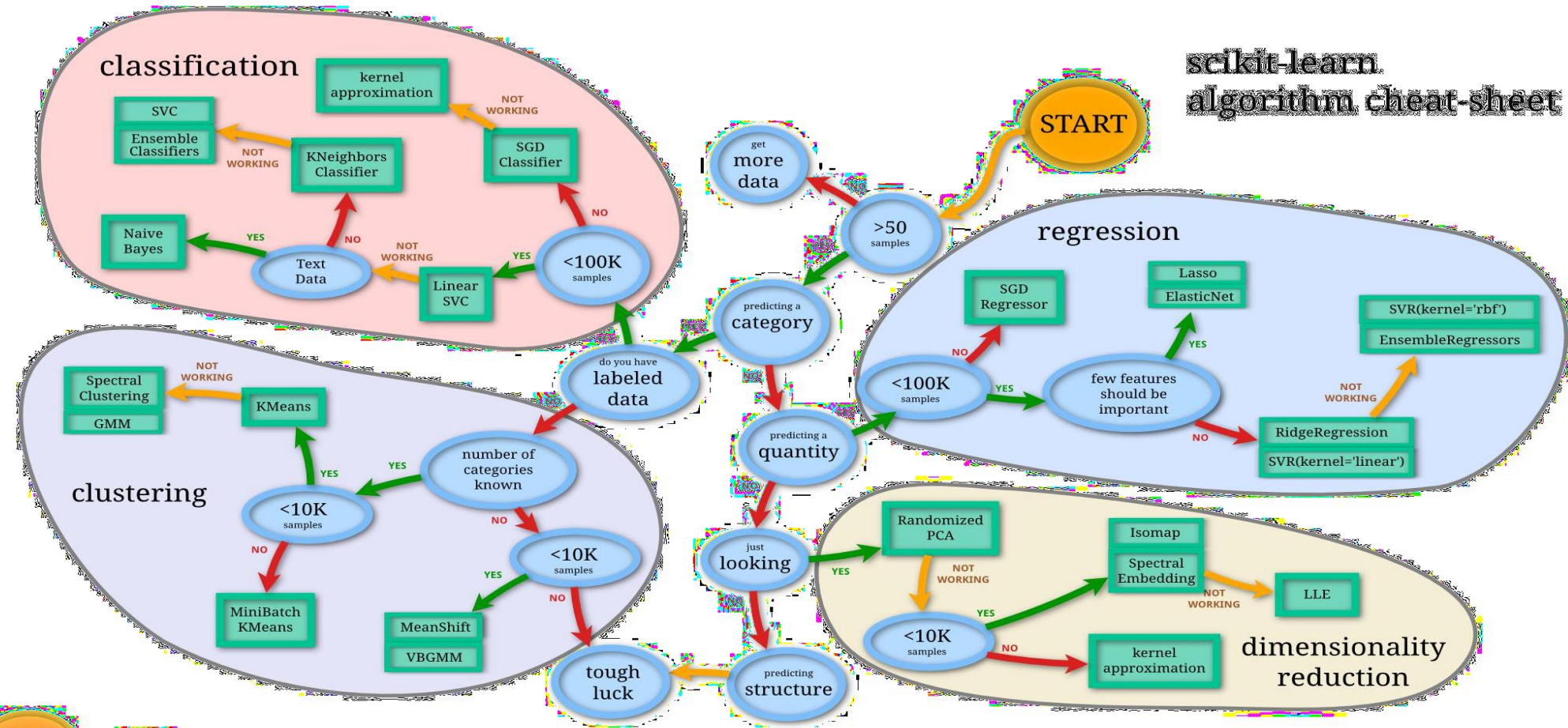
```
from matplotlib.patches import Ellipse
def draw_ellipse(position, covariance, ax=None, **kwargs):
    """Draw an ellipse with a given position and covariance"""
    ax = ax or plt.gca()
    # Convert covariance to principal axes
    if covariance.shape == (2, 2):
        U, s, Vt = np.linalg.svd(covariance)
        angle = np.degrees(np.arctan2(U[1, 0], U[0, 0]))
        width, height = 2 * np.sqrt(s)
    else:
        angle = 0
        width, height = 2 * np.sqrt(covariance)
    # Draw the Ellipse
    for nsig in range(1, 4):
        ax.add_patch(Ellipse(position, nsig * width, nsig * height,
                              angle, **kwargs))

def plot_gm(gmm, X, label=True, ax=None):
    ax = ax or plt.gca()
    labels = gmm.fit(X).predict(X)
    if label:
        ax.scatter(X[:, 0], X[:, 1], c=labels, s=40, cmap='viridis', zorder=2)
    else:
        ax.scatter(X[:, 0], X[:, 1], s=40, zorder=2)
    ax.axis('equal')

    w_factor = 0.2 / gmm.weights_.max()
    for pos, covar, w in zip(gmm.means_, gmm.covariances_, gmm.weights_):
        draw_ellipse(pos, covar, alpha=w * w_factor)

plot_gm(gm, X)
```


9. Choosing the right estimator



3. Save and Load Machine Learning Models

```
from sklearn.externals import joblib
```

```
# Save the model as a pickle in a file
```

```
joblib.dump(my_model, 'filename.pkl')
```

```
# Load the model from the file
```

```
my_model_from_joblib = joblib.load('filename.pkl')
```

```
# Use the loaded model to make predictions
```

```
my_model_from_joblib.predict(X_test)
```

Thank you for your
attention

BEDAD Fatima...